Are Enums Necessary

Is there a way to achieve what was achieved using Enums in the previous program? Yes, using macros as shown below:

```
# include <string.h>
# define ASSEMBLY 0
# define MANUFACTURING 1
# define ACCCOUNTS 2
# define STORES 3

void main( )
{
    struct employee
    {
        char name[30] ;
        int age ;
        float bs ;
        int department ;
    } ;
    struct employee e ;
    strcpy ( e.name, "Lothar Mattheus" ) ;
    e.age = 28 ;
    e.bs = 5575.50 ;
    e.department = MANUFACTURING ;
}
```

If the same effect—convenience and readability can be achieved using macros then why should we prefer enums? Because, macros have a global scope, whereas, scope of enum can either be global (if declared outside all functions) or local (if declared inside a function).

Renaming Data types with *typedef*

There is one more technique, which, in some situations, can help to clarify the source code of a C program. This technique is to

make use of the **typedef** declaration. Its purpose is to redefine the name of an existing variable type.

For example, consider the following statement in which the type **unsigned long int** is redefined to be of the type **TWOWORDS**:

```
typedef unsigned long int TWOWORDS ;
```

Now we can declare variables of the type **unsigned long int** by writing,

```
TWOWORDS var1, var2 ;
```

instead of

```
unsigned long int var1, var2 ;
```

Thus, **typedef** provides a short and meaningful way to call a data type. Usually, uppercase letters are used to make it clear that we are dealing with a renamed data type.

While the increase in readability is probably not great in this example, it can be significant when the name of a particular data type is long and unwieldy, as it often is with structure declarations. For example, consider the following structure declaration:

```
struct employee
{
    char name[30] ;
    int age ;
    float bs ;
} ;
struct employee e ;
```

This structure declaration can be made more handy to use when renamed using **typedef** as shown below: